

Problem Definition

In the Drop-In Model of ASAP, we define best-effort-manner sketch-aware SQL rewriting and middle box for approximate query engine, with an exact backend database storing all raw tuples and base relations R.

Goal: Enable sketch operators in SQL

Making sketches of first-class SQL operators by SQL query rewriting aware of sketches' semantics, error bounds, and costs (including precomputation or not), to achieve query semantic equivalence, bounded final query errors, and optimal query costs.

Design:

1. **Analysis by SQL operators: Starting by studying the relationship between newly introduced sketch operators and existing SQL operators, for semantic equivalent SQL rewriting.**

Schema of summary (sketch): table of sketches in relational table, plus a column of what statistic is represented by a sketch column.

High-level idea: Pushing down not sketchable operators by building more lower-level sketches in AST.

SQL Operators	Mapping to Sketch
Aggregation and Group By SELECT k, $\alpha(b)$ FROM x GROUP BY x.k	α = supported aggregation by a sketch type Provide 1 sketch per k SELECT x.k, Sketch_ α (x.b) AS sk FROM x WHERE x.k = v GROUP BY x.k;
Projection with Aggregation SELECT k_1, ..., k_n, $\alpha(b)$ From R x GROUP BY k_1, ..., k_n	Mapping 1: Per projection tuple, maintain a sketch of α SELECT k_1, ..., k_n, Sketch_ α (b) AS sk FROM R AS x GROUP BY k_1, ..., k_n; Mapping 2: Sketch of sketches (Hydra) to sketch multiple subpopulations WITH fine AS (

	<pre> SELECT k_1, ..., k_n, Hydra_Sketch_α(b) AS sk FROM R AS x GROUP BY k_1, ..., k_n) SELECT k_1, ..., k_n, MERGE(sk) -- merge sketches across the rows that share R.k_i AS sk_hydra FROM fine GROUP BY k_1, ..., k_n; </pre>
Selection (filter) with Aggregation <pre> SELECT k_1, ..., k_n, α(b) FROM R AS x GROUP BY k_1, ..., k_n HAVING p(k_1, ..., k_n); </pre>	Solution 1: keep Mapping 1, apply selection on keys after <pre> WITH g AS (SELECT k_1, ..., k_n, SKETCH_ALPHA(b) AS sk FROM R AS x GROUP BY k_1, ..., k_n) SELECT * FROM g WHERE p(k_1, ..., k_n); </pre> Solution 2: Pushing down the selection operator before projection, and then mapping 1 or 2. <pre> SELECT k_1, ..., k_n, SKETCH_ALPHA(b) AS sk FROM R AS x WHERE p_tuple(x) -- predicate over base tuples GROUP BY k_1, ..., k_n; </pre>
Join with Aggregation <pre> WITH orders_agg AS (SELECT customer_id, SUM(amount) AS total_orders FROM orders GROUP BY customer_id), payments_agg AS (SELECT customer_id, SUM(amount) AS total_payments FROM payments GROUP BY customer_id) </pre>	pre-aggregate sketches on the side being summarized by the join key, then join. <pre> WITH S_sk AS (SELECT id, α(b) AS sk FROM S GROUP BY id) SELECT R.k, MERGE(sk) -- merge sketches across the rows </pre>

SELECT o.customer_id, o.total_orders, p.total_payments FROM orders_agg o JOIN payments_agg p ON o.customer_id = p.customer_id;	that share R.k FROM R JOIN S_sk ON R.id = S_sk.id GROUP BY R.k;
Distinct (explicitly deduplicate) with Aggregation	Any sketch where duplicates should not change the state; Maintain a separate sketch for deduplicated tuples (pushing down distinct) WITH d AS (SELECT DISTINCT x FROM R) SELECT SKETCH_ALPHA(x) AS sk FROM d;
Set/bag with Aggregation SELECT $\alpha(x)$ FROM R UNION ALL SELECT $\alpha(x)$ FROM S;	$\alpha(R \uplus S) = \alpha(R) \oplus \alpha(S)$ requires mergeability of sketches Sketch then merge: WITH r_sk AS (SELECT SKETCH_ALPHA(x) AS sk FROM R), s_sk AS (SELECT SKETCH_ALPHA(x) AS sk FROM S) SELECT MERGE_ALL(sk) AS sk FROM (SELECT sk FROM r_sk UNION ALL SELECT sk FROM s_sk) t;

Pseudocode for identifying feasibility of mapping a SQL to SQL-Sketch

Step 1: SQL->AST_SQL

Step 2: Traverse AST_SQL for generating feasible AST_Sketch s

Algorithm: Feasible AST_Sketch s generation

Input: one AST_SQL

Output: List of feasible AST_Sketches

Assuming in AST, Node: data, Edge: operator

Func AST_SQL_to_sketch(AST_SQL):

// recursive, and replacement from SQL aggregation function to sketch is from leaf layer to root

e = Find a “**FUNCTION**” edge in AST_SQL;

Aggregation = find aggregation node under FUNCTION

Project_node = Find **e**’s “**PROJECT**” ancestor node

Group_by_check = Find whether **project_node** has siblings and ancestors of “**GROUP BY**” nodes and its parameter columns <- can be NULL

Where_check = find project_node’s siblings and ancestors of “**WHERE**” node and its parameter columns

Join_check = find if any JOIN in the path from e to project_node

Subtree_SQL = AST_subtree(e)

Replaced_subtree= Try_Replace_subtree_SQL_to_Sketch(Subtree_SQL, group_by_node, where_node, Join_check, Aggregation)

AST_sketch = replace Subtree_SQL to replaced_subtree for existing AST_SQL

AST_SQL_to_sketch(e)

List_AST_Sketches.append(AST_sketch)

Return List of AST_Sketches

Func Try_Replace_subtree_SQL_to_Sketch(subtree_SQL, Group_by_node, where_node, Join_check, Aggregation):

Sketch_type_candidates = Look up Generate_SQL_Aggregation_Sketch_Mapping(Aggregation) <- e.g., a CMSketch, CS

Bool check = Check if Sketch_type_candidates preserve the Group_by_node, where_node, join_check where filters

If check == true:

subtree_ASAP = Generate by replacing Aggregation node with the Sketch “node”

Return subtree_ASAP

Else:

Actions = {

Group_by_preserving,

Where_preserving,

Join_push_down,

// More...

}

If Group_by_preserving or where_preserving:

subtree_ASAP = Don’t replace this aggregation function with sketch (may not be optimal)

If Join_push_down:
 subtree_ASAP = Reconstruct subtree_SQL with join (group_by key) push down, replace the aggregation to per group_by key, sketch, instead of a sketch for all keys.

Return subtree_ASAP

Func Generate_SQL_Aggregation_Sketch_Mapping()
 // Rule based, or TBD

Return A mapping from SQL aggregators to a list of feasible sketches (assume generated by another module)

Func AST_subtree(AST, a node or an edge to its father in this AST):
Return the subtree data structure with this node as the subtree root

2. Searching combination of sketches for 1 query result's accuracy SQL-to-Sketch translation
 - a. Including sketch error bound analysis
3. Finding optimal query plan with Sketch operators-in-SQL
 - a. Cost-based SQL query optimizer extension with consideration of
 - i. Sketch costs
 - ii. Precomputation or not
 - iii. Middle box query model

ClickBench case study

SQL	Sketch-mapping based on AST
SELECT COUNT(*) FROM hits;	N/A

SELECT UserID FROM hits WHERE UserID = 435090932899640449;	
SELECT COUNT(*) FROM hits WHERE AdvEngineID <> 0;	1 query solution: Maintain per AdvEngineID count. Mapping: A frequency sketch for AdvEngineID Keys.
SELECT SUM(AdvEngineID), COUNT(*), AVG(ResolutionWidth) FROM hits;	1 query solution: There is no group-by and the query is simple counting over each column. No sketch is needed for one sum, avg, and count aggregator.
SELECT AVG(UserID) FROM hits;	
SELECT COUNT(DISTINCT UserID) FROM hits;	1 query solution: A distinct counting sketch for the queried column.
SELECT COUNT(DISTINCT SearchPhrase) FROM hits;	
SELECT MIN(EventDate), MAX(EventDate) FROM hits;	1 query solution: Maintain min, max for EventDate column.
SELECT AdvEngineID, COUNT(*) FROM hits WHERE AdvEngineID <> 0 GROUP BY AdvEngineID ORDER BY COUNT(*) DESC;	1 query solution: Maintain a frequency sketch for one column.
SELECT SearchPhrase, COUNT(*) AS c FROM hits WHERE SearchPhrase <> " GROUP BY SearchPhrase ORDER BY c DESC LIMIT 10;	
SELECT UserID, COUNT(*) FROM hits GROUP BY UserID ORDER BY COUNT(*) DESC LIMIT 10;	
SELECT COUNT(*) FROM hits WHERE URL LIKE '%google%';	
SELECT RegionID, COUNT(DISTINCT UserID) AS u FROM hits GROUP BY RegionID ORDER BY u DESC LIMIT 10;	1 query solution: A super spider sketch for RegionID and UserID; Or one distinct counting sketch per RegionID value.
SELECT MobilePhoneModel, COUNT(DISTINCT UserID) AS u FROM hits WHERE MobilePhoneModel <> " GROUP BY MobilePhoneModel ORDER BY u DESC LIMIT 10;	
SELECT SearchPhrase, COUNT(DISTINCT UserID) AS u FROM hits WHERE SearchPhrase	

<> " GROUP BY SearchPhrase ORDER BY u DESC LIMIT 10;	
SELECT RegionID, SUM(AdvEngineID), COUNT(*) AS c, AVG(ResolutionWidth), COUNT(DISTINCT UserID) FROM hits GROUP BY RegionID ORDER BY c DESC LIMIT 10;	1 CountSketch for AdvEngineID column, 1 frequency sketch for RegionID, 1 CountSketches for sum of ResolutionWidth, 1 distinct counting sketch per RegionID.
SELECT MobilePhone, MobilePhoneModel, COUNT(DISTINCT UserID) AS u FROM hits WHERE MobilePhoneModel <> " GROUP BY MobilePhone, MobilePhoneModel ORDER BY u DESC LIMIT 10;	1 query solution: - per <MobilePhone, MobilePhoneModel> pair, one distinct counting, or frequency counting sketch. - Create a sketch of sketches, e.g., Hydra
SELECT SearchEngineID, SearchPhrase, COUNT(*) AS c FROM hits WHERE SearchPhrase <> " GROUP BY SearchEngineID, SearchPhrase ORDER BY c DESC LIMIT 10;	
SELECT UserID, SearchPhrase, COUNT(*) FROM hits GROUP BY UserID, SearchPhrase ORDER BY COUNT(*) DESC LIMIT 10;	
SELECT UserID, SearchPhrase, COUNT(*) FROM hits GROUP BY UserID, SearchPhrase LIMIT 10;	
SELECT UserID, extract(minute FROM EventTime) AS m, SearchPhrase, COUNT(*) FROM hits GROUP BY UserID, m, SearchPhrase ORDER BY COUNT(*) DESC LIMIT 10;	
SELECT SearchPhrase, MIN(URL), COUNT(*) AS c FROM hits WHERE URL LIKE '%google%' AND SearchPhrase <> " GROUP BY SearchPhrase ORDER BY c DESC LIMIT 10;	A frequency sketch for SearchPhrase col, a distinct counting for UserID per SearchPhrase. a min of URL for each SearchPhrase entry. -> Sketch partially solves it.
SELECT SearchPhrase, MIN(URL), MIN(Title), COUNT(*) AS c, COUNT(DISTINCT UserID) FROM hits WHERE Title LIKE '%Google%' AND URL NOT LIKE '%.google.%' AND SearchPhrase <> " GROUP BY SearchPhrase ORDER BY c DESC LIMIT 10;	

Backup

SQL Query Execution Order

The general logical order of SQL query execution is as follows:

1. FROM (and JOIN operations)
2. WHERE (filters individual rows before any aggregations or window calculations)
3. GROUP BY
4. HAVING (filters groups after aggregation)
5. WINDOW (optional, defines named windows for use in window functions)
6. Window Functions (calculated across the defined window of rows)
7. QUALIFY (filters rows based on the window function results)
8. SELECT
9. DISTINCT
10. ORDER BY
11. LIMIT / OFFSET



